

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab# **03**)

Apply Behavioral-level Verilog to Design and Simulate Combinational Modules

Student Name: _Eesha Rehan_____ Roll No: 24i-6027_____ Section: B__

Date performed: _19thAug_____, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1:

Description:

In this task we are required to make 4-bit adder/subtractor in which we performed it by both gate level and behavioral level. Firstly we did the simulation on the Logisim then by gate level we simply made the half adder and full adder modules so we used them in making the 4 bit adder we take 4 bit input of A and B and the output as sum as four bit and the carry then we call the half adder by index of 0 and then full adder as 1 index and so on we also make the test bench and tested it by giving adding and subtracting the bits and also did by behavioral by just using assign and doing addition and subtraction.

Code:

4BitAdder(gate level):

```
module halfadder(a,b,s,c);
input a;
input b;
output s;
output c;
xor x1(s,a,b);
and a1(c,a,b);
endmodule

module testbench();
reg a;
reg b;
wire s;
wire c;
halfadder unit(a,b,s,c);
initial
begin
    a=0; b=0;
    #50 a=0; b=1;
    #50 a=1; b=0;
    #50 a=1; b=1;
end
endmodule

module fulladder(a,b,cin,s,c);
input a;
input b;
input cin;
output s;
output c;
wire t1;//sum
wire t2;//carry
wire t3;
halfadder U1(a,b,t1,t2);
```

```
halfadder U2(t1,cin,s,t3);
or(c,t2,t3);
endmodule
module testbench();
reg a;
reg b;
reg cin;
wire s;
wire c;
fulladder unit(a,b,cin,s,c);
initial
begin
a=0;b=0;cin=0;
#50 a=0;b=0;cin=1;
#50 a=0;b=1;cin=0;
#50 a=0;b=1;cin=1;
#50 a=1;b=0;cin=0;
#50 a=1;b=0;cin=1;
#50 a=1;b=1;cin=0;
#50 a=1;b=1;cin=1;
end
endmodule
module adder4bit(A, B, Sum, Cout);
input [3:0] A, B;
output [3:0] Sum;
output Cout;
wire t1;
wire t2;
wire t3;
halfadder u1(A[0], B[0], Sum[0], t1);
fulladder u2(A[1], B[1], t1, Sum[1], t2);
fulladder u3(A[2], B[2], t2, Sum[2], t3);
fulladder u4(A[3], B[3], t3, Sum[3], Cout);
endmodule
module testbench();
reg [3:0] A, B;
wire [3:0] Sum;
wire Cout;
adder4bit unit(A, B, Sum, Cout);
initial begin
A=4'b0000; B=4'b0000;
#50 A=4'b0011; B=4'b0001;
#50 A=4'b0111; B=4'b0010;
#50 A=4'b1010; B=4'b0101;
#50 A=4'b1111; B=4'b0001;
#50 A=4'b1111; B=4'b1111;
end
```

```

endmodule

4BitAdder(behavioral level):
odule adder4bit(A,B,Sum_add,Cout_add,Diff_sub,Bout_sub);
input [3:0] A, B;
output [3:0] Sum_add;
output Cout_add;
output [3:0] Diff_sub;
output Bout_sub;
assign {Cout_add, Sum_add} = A + B;
assign {Bout_sub, Diff_sub} = A - B;
endmodule

module testbench();
reg [3:0] A, B;
wire [3:0] Sum_add, Diff_sub;
wire Cout_add;
wire Bout_sub;

adder4bit unit(A, B, Sum_add, Cout_add, Diff_sub, Bout_sub);

initial
begin
    A=4'b0000; B=4'b0000; #50;
    A=4'b0011; B=4'b0001; #50;
    A=4'b0111; B=4'b0010; #50;
    A=4'b1010; B=4'b0101; #50;
    A=4'b1111; B=4'b0001; #50;
    A=4'b1111; B=4'b1111; #50;
end
endmodule

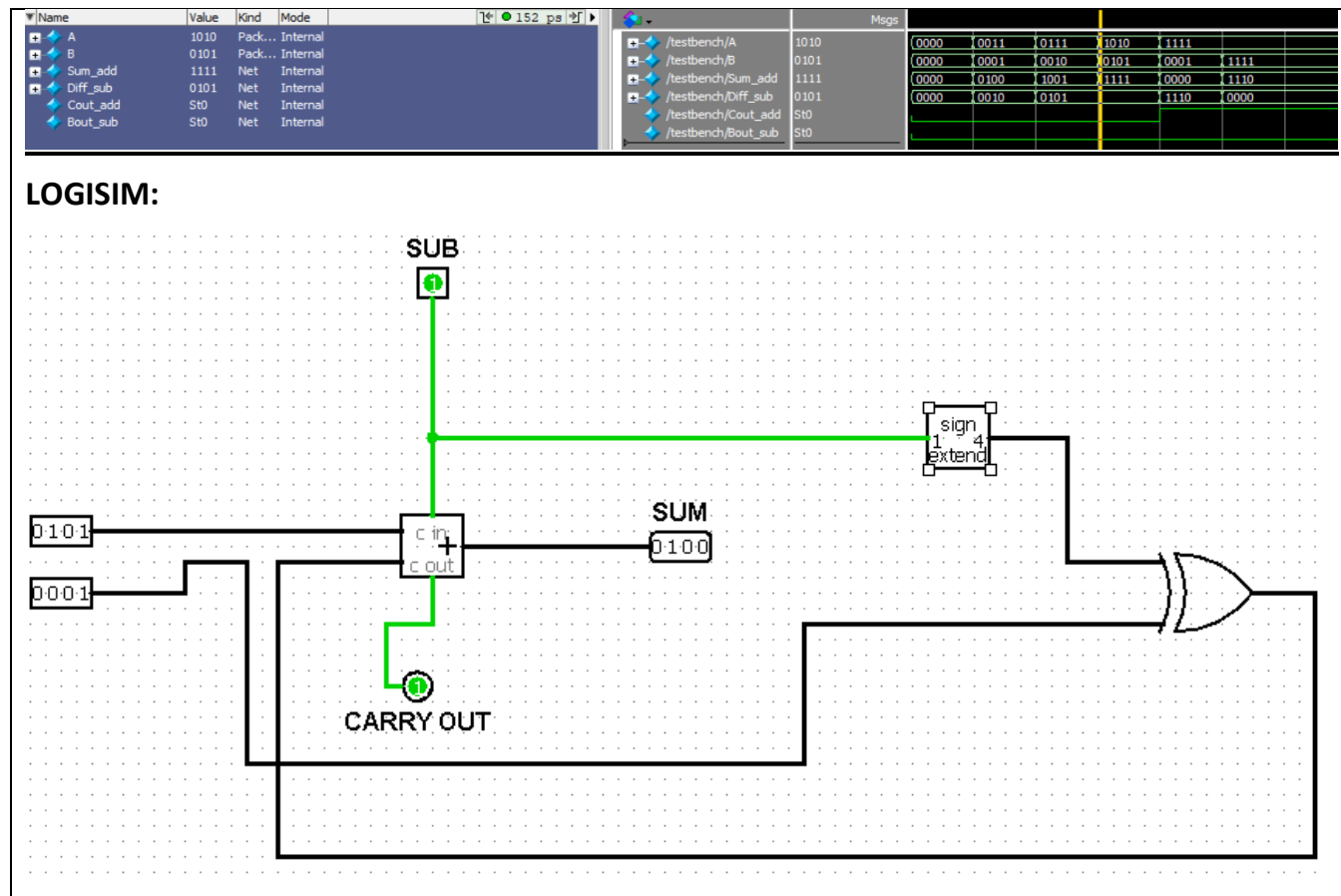
```

Output:

4bitadder/sub(gatelevel):

Name	Value	Kind	Mode	Msgs
/testbench/A	1111	Pack...	Internal	0000 0011 0111 1010 1111
/testbench/B	1111	Pack...	Internal	0000 0001 0010 0101 0001 1111
/testbench/Sum	1110	Net	Internal	0000 0100 1001 1111 0000 1110
/testbench/Cout	St1	Net	Internal	

4bitadder/subtractor(behavioral level):



LOGISIM:

Task 2:

Description:

In this task we are required to make 2x1 mux in which there is one selection line that will decide the operation if it is 0 then AND gate will be applied or if it is 1 then or gate will be applied. Two inputs are taken as A and B and a selection line then we did it by writing behavioral level code and make a test bench to test it. Also, we make simulation on Logisim for testing it.

Code:

```
module MUX_2X1(A,B,S,Y);
```

```
input A;
```

```
input B;
```

```
input S;
```

```
output Y;

    assign Y = S ? (A | B) : (A & B);

endmodule

module testbench();

reg A;

reg B;

reg S;

wire Y;

MUX_2X1 unit(A,B,S,Y);

initial

begin

    A=0; B=0; S=0;

    #50 A=0; B=1; S=0;

    #50 A=1; B=0; S=0;

    #50 A=1; B=1; S=0;

    #50 A=0; B=0; S=1;

    #50 A=0; B=1; S=1;

    #50 A=1; B=0; S=1;

    #50 A=1; B=1; S=1;

end

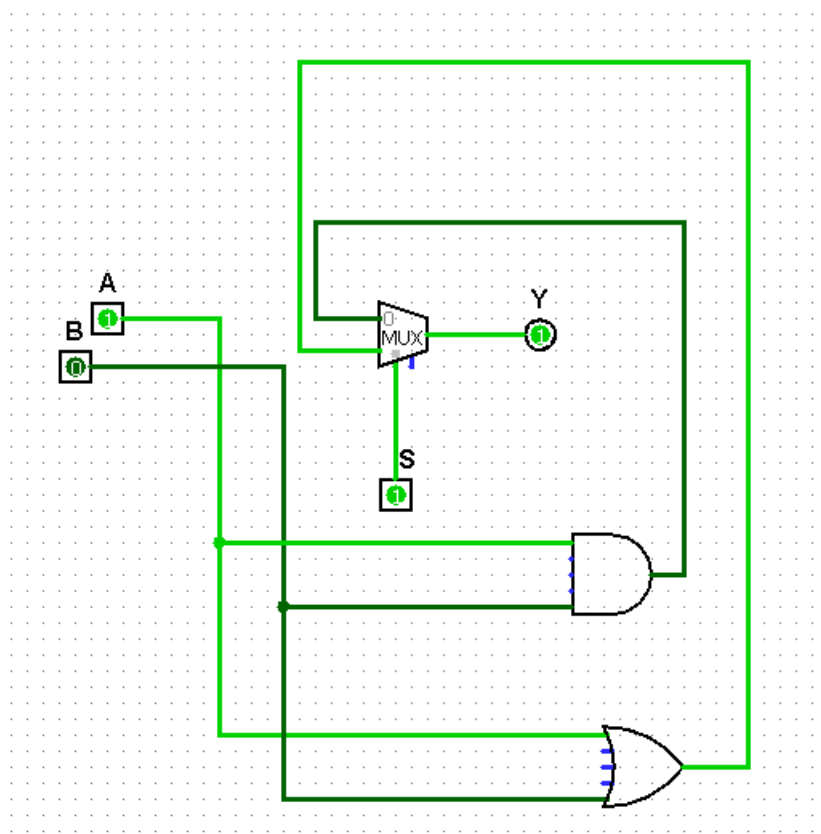
endmodule
```

Output:

MUX2X1(BEHAVIORAL LEVEL):

Objects				Wave - Default		
Name	Value	Kind	Mode		Msgs	
A	1	Regi...	Internal	/testbench/A	1	
B	1	Regi...	Internal	/testbench/B	1	
S	1	Regi...	Internal	/testbench/S	1	
Y	St1	Net	Internal	/testbench/Y	St1	

LOGISIM:



Task 3:

Description:

In this task we are required to make comparator that will do is the two input are equal greater than or less than by using continuous assignment assign then do it by behavioral level taking inputs that will be compared are taken as A and B then made a test bench that will test these inputs bits.

CODE:

```
module comparator(A, B, EQ, GT, LT);
```

```
input [1:0] A, B;
```

```
output EQ;
```

```
output GT;
```

```
output LT;
```

```
assign EQ = (A == B);
```

```
assign GT = (A > B);
```

```
assign LT = (A < B);
```

```
endmodule
```

```
module testbench();
```

```
reg [1:0] A, B;
```

```
wire EQ;
```

```
wire GT;
```

```
wire LT;
```

```
comparator unit(A, B, EQ, GT, LT);
```

```
initial
```

```
begin
```

```
A=2'b00; B=2'b00;
```

```
#50 A=2'b00; B=2'b01;
```

```
#50 A=2'b01; B=2'b00;
```

```
#50 A=2'b01; B=2'b01;
```

```
#50 A=2'b10; B=2'b01;
```

```
#50 A=2'b01; B=2'b11;
```

```
#50 A=2'b11; B=2'b11;
```

```
end
```

```
endmodule
```

Output:

Name	Value	Kind	Mode	Msgs
A	11	Pack...	Internal	
B	11	Pack...	Internal	
EQ	St1	Net	Internal	
GT	St0	Net	Internal	
LT	St0	Net	Internal	

/testbench/A	11		.00		.01		.10		.01		.11
/testbench/B	11		.00		.01		.00		.01		.11
/testbench/EQ	St1										
/testbench/GT	St0										
/testbench/LT	St0										

LOGISIM:

